

Race Condition

1. Write a C program where two processes simultaneously write to the same file. Demonstrate the race condition and suggest a solution using file locks.

CODE:

```
#include <stdio.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/wait.h>

int main() {

    int fd = open("race.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);

    if(fork() == 0) {

        for(int i=0;i<5;i++){

            write(fd,"Child\n",6);

            sleep(1); }

    } else {

        for(int i=0;i<5;i++){

            write(fd,"Parent\n",7);

            sleep(1); }

        wait(NULL);

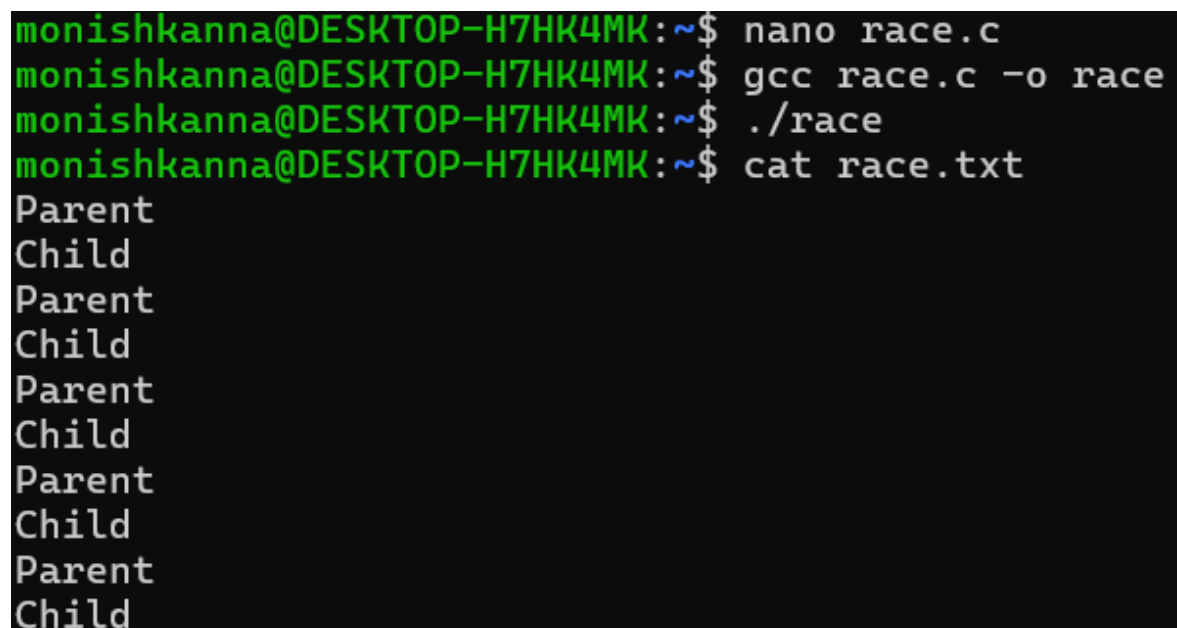
    }

    close(fd);

    return 0;

}
```

OUTPUT:



```
monishkanna@DESKTOP-H7HK4MK:~$ nano race.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc race.c -o race
monishkanna@DESKTOP-H7HK4MK:~$ ./race
monishkanna@DESKTOP-H7HK4MK:~$ cat race.txt
Parent
Child
Parent
Child
Parent
Child
Parent
Child
Parent
Child
```

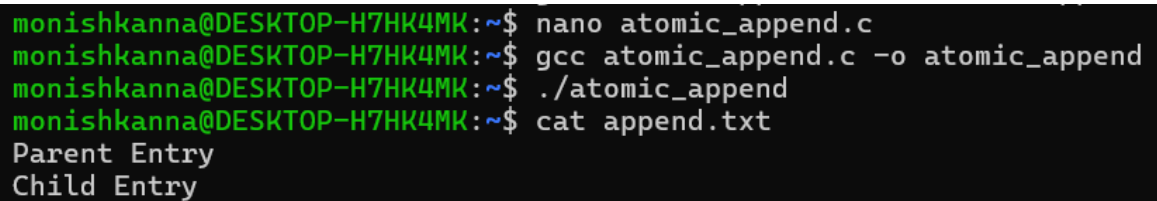
Atomicity

1. Write a C program that appends data to a file from multiple processes. How do you ensure the append operation is atomic?

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
int main() {
    int fd = open("append.txt", O_WRONLY | O_CREAT | O_APPEND, 0666);
    if(fork() == 0) {
        write(fd, "Child Entry\n", 12);
    } else {
        write(fd, "Parent Entry\n", 13);
        wait(NULL); }
    close(fd);
    return 0;
}
```

OUTPUT:



```
monishkanna@DESKTOP-H7HK4MK:~$ nano atomic_append.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc atomic_append.c -o atomic_append
monishkanna@DESKTOP-H7HK4MK:~$ ./atomic_append
monishkanna@DESKTOP-H7HK4MK:~$ cat append.txt
Parent Entry
Child Entry
```

2. Demonstrate atomic read-modify-write on a shared file using system calls. What happens if atomicity is not guaranteed?

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
void increment(int fd) {
    struct flock lock;
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;
    lock.l_pid = getpid();
    fcntl(fd, F_SETLKW, &lock);
    char c;
    lseek(fd, 0, SEEK_SET);
    read(fd, &c, 1);
    int num = c - '0';
    num++;
    c = num + '0';
    lseek(fd, 0, SEEK_SET);
    write(fd, &c, 1);
    lock.l_type = F_UNLCK;
    fcntl(fd, F_SETLK, &lock); }
```

```

int main() {
    int fd = open("counter.txt", O_RDWR | O_CREAT | O_TRUNC, 0666);
    write(fd, "0", 1);
    if(fork() == 0) {
        increment(fd);
        exit(0);
    } else {
        increment(fd);
        wait(NULL);
    }
    close(fd);
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano atomic_rmw.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc atomic_rmw.c -o atomic_rmw
monishkanna@DESKTOP-H7HK4MK:~$ ./atomic_rmw
monishkanna@DESKTOP-H7HK4MK:~$ cat counter.txt
2monishkanna@DESKTOP-H7HK4MK:~$ |

```

Pre-Emptive and Non Pre-Emptive Concept

1. Write a C program using threads to show how pre-emptive scheduling can cause unexpected results when accessing shared data.

CODE:

```

#include <stdio.h>
#include <pthread.h>
int counter = 0;
void* increment(void* arg){
    for(int i=0;i<100000;i++){
        counter++;
    }
    return NULL;
}
int main(){
    pthread_t t1,t2;
    pthread_create(&t1,NULL,increment,NULL);
    pthread_create(&t2,NULL,increment,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    printf("Final Counter: %d\n",counter);
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano preemptive.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc preemptive.c -o preemptive -lpthread
monishkanna@DESKTOP-H7HK4MK:~$ ./preemptive
Final Counter: 113130

```

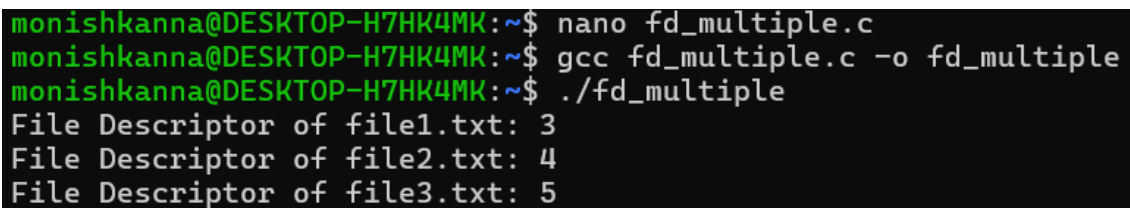
File Descriptor Table and Open File Table

1. Write a C program to open multiple files and display their file descriptors. Explain the relationship between the process's file descriptor table and the system-wide open file table.

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd1 = open("file1.txt", O_CREAT | O_RDWR, 0666);
    int fd2 = open("file2.txt", O_CREAT | O_RDWR, 0666);
    int fd3 = open("file3.txt", O_CREAT | O_RDWR, 0666);
    if(fd1 < 0 || fd2 < 0 || fd3 < 0){
        perror("Open failed");
        return 1;    }
    printf("File Descriptor of file1.txt: %d\n", fd1);
    printf("File Descriptor of file2.txt: %d\n", fd2);
    printf("File Descriptor of file3.txt: %d\n", fd3);
    close(fd1);
    close(fd2);
    close(fd3);
    return 0;
}
```

OUTPUT:



```
monishkanna@DESKTOP-H7HK4MK:~$ nano fd_multiple.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc fd_multiple.c -o fd_multiple
monishkanna@DESKTOP-H7HK4MK:~$ ./fd_multiple
File Descriptor of file1.txt: 3
File Descriptor of file2.txt: 4
File Descriptor of file3.txt: 5
```

2. Demonstrate how closing a file descriptor affects the file descriptor table and open file table. What happens if multiple descriptors refer to the same file?

CODE:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    int fd1 = open("sample.txt", O_CREAT | O_RDWR | O_TRUNC, 0666);
    if(fd1 < 0){
        perror("Open failed");
        return 1;
    }
    int fd2 = dup(fd1);
    printf("Original FD: %d\n", fd1);
    printf("Duplicated FD: %d\n", fd2);
    write(fd1, "Hello\n", 6);
    close(fd1);
    printf("Closed original FD\n");
    write(fd2, "World\n", 6);
    close(fd2);
    printf("Closed duplicated FD\n");
    return 0;
}
```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano fd_close.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc fd_close.c -o fd_close
monishkanna@DESKTOP-H7HK4MK:~$ ./fd_close
Original FD: 3
Duplicated FD: 4
Closed original FD
Closed duplicated FD
monishkanna@DESKTOP-H7HK4MK:~$ cat sample.txt
Hello
World

```

Duplicating File Descriptor - dup() System Call

1. Write a C program that uses dup() to duplicate a file descriptor and show that both descriptors refer to the same open file description.

CODE:

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main(){
    int fd = open("dup.txt", O_CREAT|O_WRONLY,0666);
    int fd2 = dup(fd);
    write(fd,"Hello\n",6);
    write(fd2,"World\n",6);
    printf("FD1: %d FD2: %d\n",fd,fd2);
    close(fd);
    close(fd2);
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano dup_demo.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc dup_demo.c -o dup_demo
monishkanna@DESKTOP-H7HK4MK:~$ ./dup_demo
FD1: 3 FD2: 4

```

2. Implement a program that redirects standard output to a file using dup2(). Verify that output is written to the file instead of the console.

CODE:

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main(){
    int fd = open("output.txt", O_CREAT|O_WRONLY|O_TRUNC,0666);
    dup2(fd, STDOUT_FILENO);
    printf("This goes into file\n");
    close(fd);
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano dup2_redirect.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc dup2_redirect.c -o dup2_redirect
monishkanna@DESKTOP-H7HK4MK:~$ ./dup2_redirect
monishkanna@DESKTOP-H7HK4MK:~$ cat output.txt
This goes into file

```